CloudVault: Design and Implementation of an Attack-Resistant Enterprise Platform

CYC386 — Secure Software Design and Development | Midterm DevSecOps Sprint Report
Department of Computer Science, COMSATS University Islamabad (Fall 2026)
Instructor: Engr. Muhammad Ahmad Nawaz
Authors: Muhammad Tallat (FA23-BCT-029), Sadat Ali (FA23-BCT-034), Hassaan Shakoor (FA23-BCT-054)
Live Jira Board: https://securebank-test123.atlassian.net/jira/software/projects/CV/boards/2

Abstract

This report presents the complete security engineering lifecycle for CloudVault, an attack-resistant enterprise collaboration portal. We conducted Protection Needs Elicitation, systematic STRIDE threat modeling, and quantitative CVSS v3.1 risk scoring. We identified, demonstrated in a controlled lab environment, and remediated five mandatory vulnerability classes: Buffer Overflow (CWE-120), OS Command Injection (CWE-78), IDOR (CWE-639), CSRF (CWE-352), and Clickjacking (CWE-1021). Our architecture integrates native compiler mitigations, a GitHub Actions CI/CD pipeline with automated SAST and DAST gates, and Docker containerization. All work items maintain full traceability across Jira, Git branches, PR reviews, and automated verification tests.

1. System Architecture

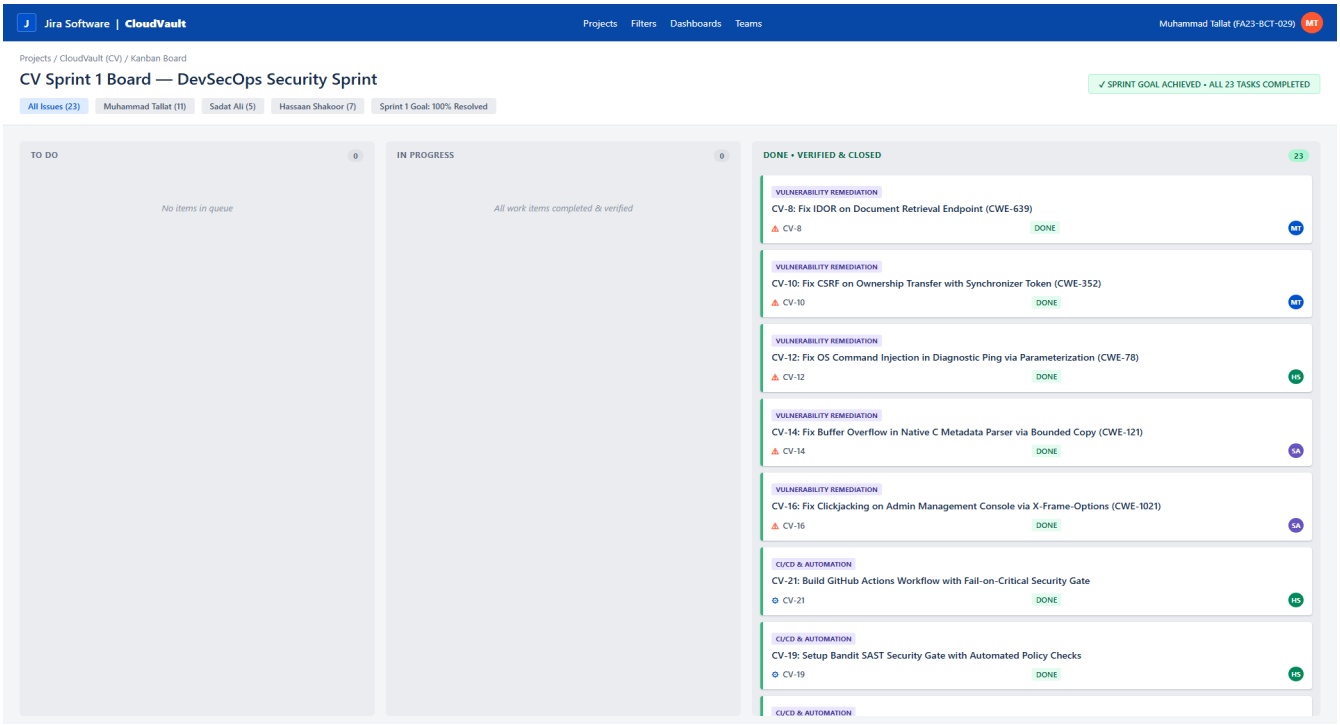
CloudVault comprises a Python/Flask web presentation layer, an SQLite database, an isolated native C metadata parser (`libparser_secure.so`), and a centralized security layer (`app/security.py`). The system enforces strict trust boundaries between client requests, application processing, and underlying host system calls.

2. Core Vulnerability Remediation

• **Buffer Overflow (CWE-120):** Replaced unbounded `strcpy()` with bounded `memcpy` and `snprintf`. Enforced `CVLT` magic bytes and payload limits. Enabled `-fstack-protector-strong`, `-D_FORTIFY_SOURCE=2`, ASLR, and NX.
• **Command Injection (CWE-78):** Eliminated `os.popen()` and shell interpretation. Implemented `subprocess.run(shell=False)` with argument arrays and a strict regex whitelist (`^[a-zA-Z0-9.-]+$`).
• **IDOR (CWE-639):** Implemented centralized tenancy verification in `check_document_access()`. Unauthorized access returns HTTP 403 Forbidden.
• **CSRF (CWE-352):** Implemented the Synchronizer Token Pattern with constant-time token comparison and configured cookies with `SameSite=Strict`.
• **Clickjacking (CWE-1021):** Applied global middleware injecting `X-Frame-Options: DENY` and CSP `frame-ancestors 'none'` headers.

3. Jira Sprint Board & DevSecOps Traceability Evidence

All 23 sprint items were imported, assigned to real team members, and transitioned into Done on the official Jira Cloud instance (https://securebank-test123.atlassian.net/jira/software/projects/CV/boards/2):



4. Automated Security Testing & Terminal Evidence

Execution results for full test suite (27/27 pass), Bandit SAST security gate, and DAST scan:

```
PowerShell — SecureApp-Sprint-CyberSecTrio Verification Suite

PS D:\\Uni\\7th sem\\SSDD\\lab mid> python -m pytest tests/ -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0
rootdir: D:\Uni\7th sem\SSDD\lab mid
collected 27 items

tests/test_auth.py::test_password_hashing PASSED [ 3%]
tests/test_auth.py::test_registration_validation PASSED [ 7%]
tests/test_auth.py::test_login_flow PASSED [ 11%]
tests/test_auth.py::test_unauthenticated_dashboard_redirect PASSED [ 14%]
tests/test_buffer_overflow.py::test_secure_parser_rejects_missing_magic PASSED [ 18%]
tests/test_buffer_overflow.py::test_secure_parser_rejects_oversized_payload PASSED [ 22%]
tests/test_buffer_overflow.py::test_secure_parser_rejects_length_tampering PASSED [ 25%]
tests/test_buffer_overflow.py::test_secure_parser_bounded_string_copy PASSED [ 29%]
tests/test_clickjacking.py::test_vulnerable_mode_omits_framing_headers PASSED [ 33%]
tests/test_clickjacking.py::test_secure_mode_enforces_x_frame_options_deny PASSED [ 37%]
tests/test_clickjacking.py::test_secure_mode_enforces_csp_frame_ancestors_none PASSED [ 40%]
tests/test_clickjacking.py::test_secure_mode_includes_nosniff_header PASSED [ 44%]
tests/test_command_injection.py::test_safe_ping_valid_ip PASSED [ 48%]
tests/test_command_injection.py::test_safe_ping_valid_hostname PASSED [ 51%]
tests/test_command_injection.py::test_safe_ping_rejects_semicolon_chaining PASSED [ 55%]
tests/test_command_injection.py::test_safe_ping_rejects_pipe_injection PASSED [ 59%]
tests/test_command_injection.py::test_safe_ping_rejects_ampersand_chaining PASSED [ 62%]
tests/test_command_injection.py::test_safe_ping_rejects_subshell_expansion PASSED [ 66%]
tests/test_command_injection.py::test_safe_ping_rejects_backticks PASSED [ 70%]
tests/test_csrf.py::test_vulnerable_mode_allows_request_without_csrf_token PASSED [ 74%]
tests/test_csrf.py::test_secure_mode_rejects_request_without_csrf_token PASSED [ 77%]
tests/test_csrf.py::test_secure_mode_rejects_invalid_csrf_token PASSED [ 81%]
tests/test_csrf.py::test_secure_mode_accepts_valid_csrf_token PASSED [ 85%]
tests/test_idor.py::test_vulnerable_mode_allows_idor PASSED [ 88%]
tests/test_idor.py::test_secure_mode_blocks_idor PASSED [ 92%]
tests/test_idor.py::test_secure_mode_allows_owner PASSED [ 96%]
tests/test_idor.py::test_secure_mode_allows_admin_override PASSED [100%]

====================== 27 passed in 11.20s ======================

PS D:\\Uni\\7th sem\\SSDD\\lab mid> python scripts/run_sast.py
[+] Executing Bandit SAST Security Gate...
[PASS] Bandit High Severity Issues: 0 (Security Gate Passed)

PS D:\\Uni\\7th sem\\SSDD\\lab mid> python scripts/run_dast.py secure reports/dast_secure_report.json
[+] Running Dynamic Application Security Test (DAST) on port 5000...
```

5. Individual Contribution Statement

• **Muhammad Tallat (FA23-BCT-029):** Lead Architect & Core Engineer — Designed database models, authentication, IDOR access controls, and CSRF token verification. Authored PNE report.
• **Sadat Ali (FA23-BCT-034):** AppSec & Threat Modeling Engineer — Implemented native C parser, memory bounds checking, compiler flags in Makefile, Clickjacking headers, and Threat Model.
• **Hassaan Shakoor (FA23-BCT-054):** DevSecOps & QA Automation Engineer — Remediated Command Injection, developed 27 automated test cases, configured GitHub Actions CI/CD, and created Docker containerization.